

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x08 实验报告

VGA 显示输出与图形界面

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 实验概述

本实验的目标是在 bootloader 中获取 UEFI 图形输出协议（GOP，Graphics Output Protocol）提供的显示信息，并将其传递给内核，进而在内核中实现一个基于线性帧缓冲（Linear Framebuffer）的 VGA 图形输出驱动。在此基础上，利用 `embedded-graphics` 等库实现字符渲染与 Shell 显示，并激活键盘中断驱动，实现通过 QEMU GUI 界面的输入输出交互。

整个实验的核心流程可以概括为以下几步：

1. 在 bootloader 中，通过 `get_handle_for_protocol::<GraphicsOutput>()` 获取 GOP 信息（帧缓冲地址、分辨率、像素格式等），并传递给内核
2. 在内核中，基于 GOP 信息初始化 VGA 帧缓冲驱动，支持逐像素绘制、矩形填充、清屏、滚动等操作
3. 利用 `embedded-graphics` 库实现字符渲染，将内置的 8×16 位图字体绘制到帧缓冲上
4. 实现一个 VGA 文本控制台，支持字符输出、换行、滚屏、回滚缓冲区等功能，并将其对接到日志系统
5. 激活键盘中断，通过共享输入缓冲区（与串口共用）的方式响应键盘输入

2. 在 Bootloader 中获取图形输出信息

2.1. UEFI Graphics Output Protocol（GOP）

UEFI 固件提供了一套标准的协议（Protocol）供 bootloader 使用，其中之一就是 Graphics Output Protocol（GOP）。GOP 可以为 bootloader 提供一个线性的帧缓冲（Linear Framebuffer），使得我们可以直接通过内存写入的方式控制屏幕上的每一个像素。

GOP 协议提供了以下关键信息：

- `frame_buffer()`：帧缓冲的起始物理地址和总大小（以字节为单位）
- `current_mode_info()`：当前显示模式的信息，包括：
  - `resolution()`：屏幕分辨率，返回 `(width, height)` 元组，单位为像素
  - `stride()`：每一行像素占用的像素数（可能大于 `width * bytes_per_pixel`，因为存在对齐填充）
  - `pixel_format()`：像素格式，常见值为 `Rgb`（红绿蓝）、`Bgr`（蓝绿红）、`Bitmask` 等

2.2. `get_gop_info()` 函数的实现

我们在 bootloader（`crates/boot/src/main.rs`）中添加了 `get_gop_info()` 函数，其实现如下：

```rust
/// Query UEFI Graphics Output Protocol for framebuffer information
fn get_gop_info() -> Option<crate::GopInfo> {
    let handle = match uefi::boot::get_handle_for_protocol::<GraphicsOutput>() {
        Ok(h) => h,
        Err(e) => {
            warn!("Failed to get GraphicsOutput handle: {:?}", e);
            return None;
        }
    };

    let mut gop = match
uefi::boot::open_protocol_exclusive::<GraphicsOutput>(handle) {
        Ok(g) => g,
        Err(e) => {
            warn!("Failed to open GraphicsOutput protocol: {:?}", e);
            return None;
        }
    };

    let mode_info = gop.current_mode_info();
    let fb = gop.frame_buffer();

    let fb_addr = gop.frame_buffer().as_mut_ptr() as u64;

    // Determine pixel format
    let format = match mode_info.pixel_format() {
        PixelFormat::Rgb => 0,
        PixelFormat::Bgr => 1,
        PixelFormat::Bitmask => 2,
        PixelFormat::BltOnly => 3,
    };

    let info = crate::GopInfo {
        fb_addr,
        fb_size: gop.frame_buffer().size() as u64,
        width: mode_info.resolution().0 as u32,
        height: mode_info.resolution().1 as u32,
        stride: mode_info.stride() as u32,
        format,
    };

    info!(
        "GOP Info: {}x{}, stride={}, fb=0x{:x}, fmt={}",
        info.width, info.height, info.stride, info.fb_addr, info.format
    );

    Some(info)
}
```

具体而言，有以下五个步骤：

1. **获取协议句柄**：调用 `uefi::boot::get_handle_for_protocol::<GraphicsOutput>()` 查找系统中支持 GOP 的硬件句柄。该函数返回 `Result<Handle, Error>`，失败则说明当前平台不支持图形输出（例如纯串口环境），此时返回 `None`。
2. **打开协议**：通过 `uefi::boot::open_protocol_exclusive::<GraphicsOutput>(handle)` 以独占方式打开 GOP 协议。这会返回一个 GOP 对象的互斥引用，保证在 UEFI 环境下不会与其他代码产生竞争。打开协议意味着我们获得了对显卡帧缓冲的控制权。
3. **读取模式信息**：`gop.current_mode_info()` 返回当前显示模式的元信息。我们从中提取分辨率（`resolution()`）、像素格式（`pixel_format()`）和每行步长（`stride()`）。
4. **读取帧缓冲地址**：`gop.frame_buffer()` 返回帧缓冲的物理地址和大小。我们通过 `.as_mut_ptr() as u64` 将指针转换为物理地址数字，供内核后续进行物理地址到虚拟地址的映射。
5. **封装为 `GopInfo` 结构体**：将所有信息填入 `GopInfo` 结构体中返回。

2.3. `GopInfo` 与 `BootInfo` 结构体

我们在 `crates/boot/src/lib.rs` 中定义了 `GopInfo` 结构体，用于在 bootloader 与内核之间传递图形信息：

```rust
/// Graphics Output Protocol info passed from bootloader to kernel
#[derive(Debug, Clone, Copy)]
pub struct GopInfo {
    /// Physical address of the framebuffer
    pub fb_addr: u64,
    /// Framebuffer size in bytes
    pub fb_size: u64,
    /// Horizontal resolution in pixels
    pub width: u32,
    /// Vertical resolution in pixels
    pub height: u32,
    /// Pixels per scanline
    pub stride: u32,
    /// Pixel format
    pub format: u32,
}
```

该结构体被嵌入到 `BootInfo` 结构体中，作为可选字段 `gop_info: Option<GopInfo>`，与其他启动信息（内存映射、内核页表、加载的应用程序等）一同传递给内核：

```rust
pub struct BootInfo<'a> {
    pub memory_map: BootMemoryMap,
    pub physical_memory_offset: u64,
    pub system_table: NonNull<core::ffi::c_void>,
    pub log_level: &'a str,
    pub loaded_apps: Option<AppList>,
    pub kernel_pages: KernelPages,
    // graphics info from GOP
    pub gop_info: Option<GopInfo>,
}
```

在 `efi_main()` 中，我们在退出 UEFI Boot Services **之前**调用 `get_gop_info()`。这是因为 GOP 协议属于 Boot Services，一旦调用 `uefi::boot::exit_boot_services()` 之后，所有 Boot Services 协议都将不可用。

具体实现如下，我们把得到的 GOP 信息放入 `bootinfo` 中，然后随着内核启动时需要的其他信息一起传给内核，

```rust
// 6.5 Query GOP info before exiting boot services
let gop_info = get_gop_info();

// 7. Exit boot and jump to ELF entry
info!("Exiting boot services...");

let mmap = unsafe { uefi::boot::exit_boot_services(None) };

// construct BootInfo
let bootinfo = BootInfo {
    memory_map: mmap.entries().copied().collect(),
    physical_memory_offset: config.physical_memory_offset,
    system_table,
    log_level: config.log_level,
    loaded_apps: apps,
    kernel_pages,
    gop_info,        // Get GOP for kernel
};

// align stack to 8 bytes
let stacktop = stack_start + stack_size * 0x1000 - 8;

jump_to_entry(&bootinfo, stacktop);
```

3. VGA 帧缓冲驱动的实现

3.1. `VgaDriver` 结构体

VGA 驱动定义在 `crates/kernel/src/drivers/vga.rs` 中, 我们使用原始指针 `*mut u8` 访问帧缓冲内存。这里的 `cursor_x` 和 `cursor_y` 参数表示当前输入光标所在的位置，为了方式多线程写导致混乱，采用了 `AtomicU32` 原子类型。

```
pub struct VgaDriver {
    /// Virtual address of the framebuffer (byte-level access)
    fb: *mut u8,
    /// Framebuffer size in bytes
    fb_size: u64,
    /// Horizontal resolution
    width: u32,
    /// Vertical resolution
    height: u32,
    /// Bytes per scanline
    stride_bytes: u32,
    /// Cursor X position (in character cells)
    cursor_x: AtomicU32,
    /// Cursor Y position (in character cells)
    cursor_y: AtomicU32,
}
```

3.2. 初始化和构造函数

然后我们将使用 `bootinfo` 传入内核的 `gop_info` 信息传入 `VgaDriver` 中，由于可能会因为内存对齐而导致每行的像素个数不是 `width * byte_per_scanline`，因此我们需要使用之前记录好的每行像素数来计算得到，从而让我们可以正确根据坐标定义得到某个输出位置对应字节的地址。

```
impl VgaDriver {
    pub unsafe fn new(gop_info: &GopInfo) -> Self {
        let fb_virt = physical_to_virtual(gop_info.fb_addr) as *mut u8;
        let stride_bytes = gop_info.stride * 4;

        info!(
            "VGA: fb_virt=0x{:x}, w={}, h={}, stride_bytes={}, fb_size={}",
            fb_virt as u64, gop_info.width, gop_info.height, stride_bytes,
gop_info.fb_size
        );

        Self {
            fb: fb_virt,
            fb_size: gop_info.fb_size,
            width: gop_info.width,
            height: gop_info.height,
            stride_bytes,
            cursor_x: AtomicU32::new(0),
            cursor_y: AtomicU32::new(0),
        }
    }
}
```

关键点：

- `gop_info.stride * 4`：GOP 的 stride 单位是像素，每个像素占 4 个 bytes（因为 QEMU 默认的 VGA caiyong1 BGRx 格式），因此字节步长为 `stride * 4`
- `fb_virt` 是帧缓冲在虚拟地址空间中的起始地址，后续所有像素写入操作都通过此指针进行

3.3. 像素格式

QEMU 默认的 VGA 帧缓冲使用 BGRx 格式（即每个像素 4 字节，从低到高依次为 Blue、Green、Red、未使用字节）。我们定义了 `Color` 类来表示像素颜色，同时我们也加入了一些颜色常量，便于绘图

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Color(u32);

impl Color {
    pub const BLACK: Self = Self(0x00000000);
    pub const WHITE: Self = Self(0x00FFFFFF);
    pub const RED: Self = Self(0x00FF0000);
    pub const GREEN: Self = Self(0x0000FF00);
    pub const BLUE: Self = Self(0x000000FF);
    // ...

    pub const fn new(r: u8, g: u8, b: u8) -> Self {
        Self(0x00000000 | ((r as u32) << 16) | ((g as u32) << 8) | (b as u32))
    }
}
```

颜色在 u32 中的布局为 `0x00RRGGBB`，虽然帧缓冲实际的字节顺序是 BGRx（即 B 在低字节，G 在中低字节，R 在中高字节），但由于我们在逐字节写入时手动编排 B、G、R 的顺序，所以 `Color` 内部使用逻辑上的 RGB 顺序存储。

3.4. 基本绘制操作

驱动提供了以下几个基本的像素级操作：

`draw_pixel`：逐像素绘制，

```rust
pub fn draw_pixel(&self, x: u32, y: u32, color: Color) {
    if x < self.width && y < self.height {
        unsafe {
            let base = (y as usize) * (self.stride_bytes as usize) + (x as usize)
* 4;
            let c = color.0;
            self.fb.add(base).write_volatile((c & 0xFF) as u8);          // B
            self.fb.add(base + 1).write_volatile(((c >> 8) & 0xFF) as u8);  // G
            self.fb.add(base + 2).write_volatile(((c >> 16) & 0xFF) as u8); // R
        }
    }
}
```

使用 `write_volatile` 而非普通写入的原因是：帧缓冲属于 MMIO 区域，编译器不能对其进行写合并或消除优化。

`fill_rect`：矩形填充，

```rust
pub fn fill_rect(&self, x: u32, y: u32, w: u32, h: u32, color: Color) {
    let x_end = (x + w).min(self.width);
    let y_end = (y + h).min(self.height);
    let c = color.0;
    let b = (c & 0xFF) as u8;
    let g = ((c >> 8) & 0xFF) as u8;
    let r = ((c >> 16) & 0xFF) as u8;

    for py in y..y_end {
        unsafe {
            let row_base = (py as usize) * (self.stride_bytes as usize);
            for px in x..x_end {
                let base = row_base + (px as usize) * 4;
                self.fb.add(base).write_volatile(b);
                self.fb.add(base + 1).write_volatile(g);
                self.fb.add(base + 2).write_volatile(r);
            }
        }
    }
}
```

`clear_screen`：清屏，即对整个屏幕以指定颜色填充，同时将光标重置到 `(0, 0)`。

`scroll_up`：向上滚动将指定像素高度的区域整体向上复制，底部以背景色填充。用于实现文本终端的滚屏效果。

3.5. `embedded-graphics` 集成

为了使 `VgaDriver` 能够与 `embedded-graphics` 生态兼容，我们实现了 `VgaDisplay` 包装器：

```rust
pub struct VgaDisplay<'a> {
    vga: &'a VgaDriver,
}

impl OriginDimensions for VgaDisplay<'_> {
    fn size(&self) -> Size {
        Size::new(self.vga.width, self.vga.height)
    }
}

impl DrawTarget for VgaDisplay<'_> {
    type Color = Color;
    type Error = core::convert::Infallible;

    fn draw_iter<I>(&mut self, pixels: I) -> Result<(), Self::Error>
    where
        I: IntoIterator<Item = Pixel<Self::Color>>,
    {
        for Pixel(Point { x, y }, color) in pixels {
            if x >= 0 && y >= 0 {
                self.vga.draw_pixel(x as u32, y as u32, color);
            }
        }
        Ok(())
    }
}
```

通过实现 `DrawTarget` trait，我们可以使用 `embedded-graphics` 提供的各种高级绘图原语（文本、形状、图像等），将绘制结果直接渲染到 VGA 帧缓冲上。

3.6. 全局实例与互斥保护

由于 VGA 帧缓冲是多核共享的硬件资源，我们使用自旋锁（`spin::Mutex`）保护对驱动的访问：

```rust
once_mutex!(pub VGA: VgaDriver);
guard_access_fn!(pub get_vga(VGA: VgaDriver));

pub fn init(gop_info: &GopInfo) {
    let driver = unsafe { VgaDriver::new(gop_info) };
    init_VGA(driver);
    get_vga_for_sure().clear_screen(Color::BLACK);
    info!(
        "VGA Driver Initialized: {}x{}",
        gop_info.width, gop_info.height
    );
}
```

`once_mutex!` 宏创建一个 `spin::Once<spin::Mutex<T>>` 全局变量，确保只初始化一次。`guard_access_fn!` 宏生成 `get_vga()`（非阻塞 try_lock）和 `get_vga_for_sure()`（阻塞式 expect）两个访问函数。

4. 利用 `embedded-graphics` 实现字符渲染

4.1. 内置 8×16 位图字体

为实现字符渲染, 我们需要一个等宽位图字体。本项目使用了一个经典的 8×16 像素的 ASCII 字体，定义在 `crates/kernel/src/drivers/font8x16.rs` 中。

字体覆盖 ASCII 可打印字符 0x20（空格）到 0x7E（波浪号），共 95 个字符。每个字符的字形数据为 16 字节（16 行，每行 8 位），编码形式为一个 `[[u8; 16]; 95]` 数组。例如，字符 `'A'`（ASCII 0x41，数组索引 `0x41 - 0x20 = 33`）的字形数据为：

```
[0x00, 0x00, 0x10, 0x38, 0x6C, 0xC6, 0xC6, 0xFE,
 0xC6, 0xC6, 0xC6, 0xC6, 0x00, 0x00, 0x00, 0x00],
```

每一位为 1 的位置表示该像素应以前景色填充，为 0 则以背景色填充。

4.2. VGA 文本控制台 `VgaConsole`

`VgaConsole`（定义在 `crates/kernel/src/drivers/vga_text.rs`）是 VGA 文本输出的核心组件。它保存了帧缓冲的引用，支持字符绘制、光标管理、行自动换行、滚屏等功能。

核心字段：

```rust
pub struct VgaConsole {
    char_width: u32,        // 字符宽度（8 像素）
    char_height: u32,       // 字符高度（16 像素）
    cols: u32,              // 每行字符数
    rows: u32,              // 行数
    fg_color: Color,        // 前景色
    bg_color: Color,        // 背景色
    cursor_x: u32,          // 光标列位置
    cursor_y: u32,          // 光标行位置
    y_offset: u32,          // 垂直像素偏移（分屏模式下非零）
    fb: *mut u8,            // 直接帧缓冲指针
    fb_width: u32,          // 帧缓冲宽度
    fb_height: u32,         // 帧缓冲高度
    fb_stride_bytes: u32,   // 每行字节数
    // 滚动回看缓冲区
    scrollback: VecDeque<String>,
    scroll_pos: u32,
    current_line: String,
    // ...
}
```

字符绘制 `draw_char_at`：

```rust
fn draw_char_at(&self, ch: char, col: u32, row: u32, fg: Color, bg: Color) {
    let px = col * self.char_width;
    let py = self.y_offset + row * self.char_height;

    // Fill character cell with background
    self.fill_rect_direct(px, py, self.char_width, self.char_height, bg);

    // Draw the character
    let idx = ch as usize;
    if idx >= 0x20 && idx <= 0x7E {
        let glyph_idx = idx - 0x20;
        if glyph_idx < FONT_8X16.len() {
            let bitmap = &FONT_8X16[glyph_idx];
            for (line, &byte) in bitmap.iter().enumerate() {
                for bit in 0..8u32 {
                    if (byte >> (7 - bit)) & 1 != 0 {
                        self.write_pixel(px + bit, py + line as u32, fg);
                    }
                }
            }
        }
    }
}
```

该函数先将字符单元格以背景色填充，然后遍历字形的每一行每一列，在位为 1 的位置写入前景像素。绘制时使用 `fb_stride_bytes` 直接寻址帧缓冲，而不是经过 `VgaDriver`（绕过互斥锁，提高性能）。

`write_char` — 单字符写入：

处理 `\n`（换行）、`\r`（回车）、`\x08`（退格）、`\t`（制表符）以及可打印字符。当光标超出右边界时自动换行，超出底部时自动向上滚动。

`write_str` 和 `fmt::Write`：

```
pub fn write_str(&mut self, s: &str) {
    for ch in s.chars() {
        self.write_char(ch);
    }
}

impl fmt::Write for VgaConsole {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.write_str(s);
        Ok(())
    }
}
```

实现了 `fmt::Write` trait 后，便可以使用 Rust 的 `write!` 和 `writeln!` 宏直接向 VGA 控制台输出格式化文本。

4.3. 滚动回看（Scrollback）机制

控制台维护了一个最多 1000 行的 `VecDeque<String>` 滚动缓冲区。所有被滚动出屏幕的文本行都会存入该缓冲区。

当用户按上/下方向键时（由键盘中断驱动调用 `scroll_console_up()` / `scroll_console_down()`），控制台会重新渲染滚动缓冲区中的历史内容：

```
pub fn render_body(&mut self) {
    let body_rows = self.rows - HEADER_ROWS;
    let n = self.scrollback.len() as u32;
    let bottom_idx = n.saturating_sub(self.scroll_pos);
    let top_idx = bottom_idx.saturating_sub(body_rows);

    self.clear_body();
    for i in top_idx..bottom_idx {
        let screen_row = HEADER_ROWS + (i - top_idx);
        if let Some(line) = self.scrollback.get(i as usize) {
            self.render_line_at(line, screen_row);
        }
    }
}
```

当 `scroll_pos == 0` 时，控制台处于“实时模式”——新输出直接绘制到屏幕上。当 `scroll_pos > 0` 时，新输出被缓冲到 `scrollback` 而不触及帧缓冲，并设置 `dirty_while_scrolled` 标志以提示用户有新的输出。

4.4. 分屏布局

`VgaConsole` 支持 `y_offset` 偏移参数，使其仅占用屏幕的下半部分。在分屏模式下：

1. 屏幕上方约 45% 的区域用于绘制旋转时钟（由 `clock_demo` 进程负责）
2. 屏幕下方约 55% 的区域用于文本控制台

控制台进一步分为两部分：

1. Header 区域（前 8 行固定）：显示 YSOS ASCII 徽标，不可滚动
2. Body 区域（剩余行）：Shell 交互 + 日志输出的可滚动区域

初始化函数 `init_vga_console_for_shell()` 在 `crates/kernel/src/drivers/vga_text.rs` 中实现，返回时钟区域的高度（像素），供 `clock_demo` 进程使用。

5. Shell 输出与日志系统对接

5.1. 日志的 VGA 输出

为了实现日志在 VGA 屏幕上的同步显示，我们修改了日志系统。在 `crates/kernel/src/utils/logger.rs` 中：

```
fn log(&self, record: &Record) {
    if self.enabled(record.metadata()) {
        // Always output to serial
        crate::utils::print_internal_no_vga(format_args!(
            "[{}] {} - {}\n\r",
            record.level(),
            record.target(),
            record.args()
        ));
        // Best-effort output to VGA (try_lock — safe from IRQ context)
        crate::drivers::vga_text::try_write_to_vga(format_args!(
            "[{}] {} - {}\n",
            record.level(),
            record.target(),
            record.args()
        ));
    }
}
```

日志同时输出到两个通道：

1. 串口：主通道，始终输出，不丢失

2. VGA 控制台：辅助通道，使用 `try_lock` 非阻塞写入。如果 VGA 控制台锁被其他进程持有，则跳过本次输出（best-effort），保证不会因日志输出导致死锁。使用 `try_lock` 的原因是日志可能在中断上下文中被调用，此时不能阻塞。

5.2. `print!` / `println!` 的双通道输出

在 `crates/kernel/src/utils/macros.rs` 中，`print_internal` 函数同时向串口和 VGA 控制台输出：

```
#[doc(hidden)]
pub fn print_internal(args: Arguments) {
    interrupts::without_interrupts(|| {
        if let Some(mut serial) = get_serial() {
            let _ = serial.write_fmt(args);
        }
        // Spin-lock VGA console (if initialized) to never drop output
        if let Some(mut console) =
crate::drivers::vga_text::get_vga_console_spin() {
            use core::fmt::Write;
            let _ = console.write_fmt(args);
        }
    });
}
```

`get_vga_console_spin()` 通过自旋等待获取锁，保证了在关中断的上下文中，输出到 VGA 的日志不会丢失。

5.3. 应用进程的 Shell

用户态的 Shell 程序（`/APP/sh`）通过系统调用进行输入输出。Shell 的 `stdin` 读取会最终调用 `input::get_line()`，后者阻塞地从共享输入缓冲区（`INPUT_BUF`）读取字符。读取到的字符同时回显到串口和 VGA 控制台。

6. 键盘驱动的实现

6.1. PS/2 键盘控制器

PS/2 键盘控制器使用 I/O 端口 `0x60`（数据端口）和 `0x64`（状态/命令端口）与 CPU 通信。驱动实现在 `crates/kernel/src/drivers/keyboard.rs` 中。

关键操作：

- 读取数据前：检查状态端口 `bit 0`（OUTPUT_FULL），为 1 表示有数据待读取
- 写入数据前：检查状态端口 `bit 1`（INPUT_FULL），为 0 表示控制器可接收命令

驱动使用 `pc-keyboard` crate 将原始扫描码（Scancode Set 1）解码为 Unicode 字符或 RawKey 事件。

6.2. 中断注册

键盘中断为 IRQ1，通过 IO APIC 路由到 CPU0。在 `crates/kernel/src/interrupt/keyboard.rs` 中：

```
pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[Interrupts::IrqBase as u8 + Irq::Keyboard as
u8].set_handler_fn(keyboard_handler);
}

pub extern "x86-interrupt" fn keyboard_handler(_st: InterruptStackFrame) {
    let scancode: u8 = unsafe
{ x86_64::instructions::port::Port::new(0x60).read() };
    let mut kb = get_ps2_keyboard_for_sure();
    kb.handle_scancode(scancode);
    super::ack(); // 向 LAPIC 发送 EOI
}
```

在 `interrupt/mod.rs` 中激活键盘 IRQ：

```
enable_irq(Irq::Keyboard as u8, 0); // enable IRQ1 for CPU0
```

6.3. 共享输入缓冲区

键盘输入和串口输入共用同一个无锁队列 `INPUT_BUF`（定义在 `crates/kernel/src/drivers/input.rs` 中）：

```
type Key = u8;

lazy_static! {
    static ref INPUT_BUF: ArrayQueue<Key> = ArrayQueue::new(128);
}

#[inline]
pub fn push_key(key: Key) {
    let _ = INPUT_BUF.push(key);
}

#[inline]
pub fn try_pop_key() -> Option<Key> {
    INPUT_BUF.pop()
}
```

`ArrayQueue` 来自 `crossbeam_queue` crate，是一个无锁的 MPSC（多生产者单消费者）队列。两个中断处理程序（键盘和串口）都可以安全地向队列推送数据。

6.4. `get_line()` 的行编辑功能

`input::get_line()` 实现了基本的行编辑功能：

- 支持退格删除（Backspace）
- 回车结束输入
- 同时向串口和 VGA 控制台回显字符

```
pub fn get_line() -> String {
    let mut serial_port = SerialPort::new(0x3F8);
    let mut line = String::with_capacity(128);
    loop {
        let ch = pop_key();
        match ch {
            b'\n' | b'\r' => {
                write!(serial_port, "\n").unwrap();
                echo_to_vga('\n');
                break;
            }
            0x08 | 0x7F => {
                if !line.is_empty() {
                    line.pop();
                    write!(serial_port, "\x08 \x08").unwrap();
                    echo_to_vga('\x08');
                }
            }
            _ => {
                if ch >= 0x20 && ch <= 0x7E {
                    line.push(ch as char);
                    write!(serial_port, "{}", ch as char).unwrap();
                    echo_to_vga(ch as char);
                }
            }
        }
    }
    line
}
```

6.5. 方向键的滚动控制

在键盘驱动的 `handle_scancode` 方法中，上/下方向键和 PageUp/PageDown 键被映射为控制台滚动操作：

```
DecodedKey::RawKey(key) => {
    match key {
        KeyCode::ArrowUp => vga_text::scroll_console_up(),
        KeyCode::ArrowDown => vga_text::scroll_console_down(),
        KeyCode::PageUp => vga_text::scroll_console_up_pages(1),
        KeyCode::PageDown => vga_text::scroll_console_down_pages(1),
        // ...
    }
}
```

这使得用户可以在 QEMU GUI 窗口中使用方向键翻阅控制台的历史输出。

7. 加分项

7.1. 键盘输入实现 IO 交互

通过上述键盘中断驱动与共享输入缓冲区的实现，我们成功实现了 QEMU GUI 窗口中的输入输出交互。用户可以直接在 QEMU GUI 窗口中：

- 看到彩色文本的 Shell 提示符
- 键入命令并看到实时回显
- 使用方向键和 PageUp/PageDown 翻看历史输出
- 所有输出同时出现在串口和 VGA 屏幕上

7.2. 分屏与动画时钟

我们实现了分屏布局：

- 屏幕上半部分（约 45%）：运行 `clock_demo` 内核线程，绘制一个实时旋转的模拟时钟
- 屏幕下半部分（约 55%）：文本控制台，包含 YSOS 标题头和可滚动的日志/Shell 输出区域

时钟的实现使用了基本的三角函数（`libm::cos` / `libm::sin`）计算指针角度，通过逐像素绘制实现了秒针、分针、时针、刻度标记和小时数字的渲染。时钟以每秒一帧的速度更新，通过 `get_time_ms()` 读取 UEFI 时间来进行帧同步。

7.3. 完整的内核启动流程

综合以上所有组件，系统的初始化流程如下（`kernel/src/lib.rs` 的 `init` 函数）：

```rust
pub fn init(boot_info: &'static BootInfo) {
    unsafe {
        uefi::table::set_system_table(boot_info.system_table.cast().as_ptr());
    }

    serial::init();             // 1. 初始化串口输出
    logger::init("Debug");      // 2. 初始化日志系统
    memory::allocator::init();// 3. 初始化内核堆分配器
    memory::address::init(boot_info);  // 4. 设置物理地址偏移
    memory::gdt::init();        // 5. 初始化 GDT
    memory::init(boot_info);  // 6. 初始化内存管理器

    if let Some(ref gop_info) = boot_info.gop_info {
        vga::init(gop_info);  // 7. 初始化 VGA 帧缓冲驱动
    }

    memory::user::init();       // 8. 初始化用户堆分配器
    interrupt::init();          // 9. 初始化中断系统（含键盘 IRQ）
    keyboard::init();           // 10. 初始化键盘驱动
    filesystem::init();         // 11. 初始化文件系统
    proc::init(boot_info);      // 12. 初始化进程管理

    x86_64::instructions::interrupts::enable();
    info!("YatSenOS initialized.");
}
```

随后 `kernel_main` 调用 `init_vga_console_for_shell()` 完成分屏设置，并创建 `clock_demo` 后台线程和 Shell 进程。

8. 原理总结

本实验的核心在于理解 UEFI GOP 协议的运作方式以及帧缓冲（Framebuffer）驱动的原理。

UEFI GOP 协议本质上为 OS 提供了一个线性帧缓冲（Linear Framebuffer），即显卡显存被直接映射到一段连续的物理地址空间中。只要向这段内存的特定偏移写入特定格式的像素数据，显卡就会在对应的屏幕位置上显示相应的颜色。这种方式简单高效，不需要与显卡驱动进行复杂交互，是现代 OS 引导阶段图形输出的标准做法。

帧缓冲的工作原理可以类比为一块画布：

1. `fb_addr` 是画布在内存中的起始地址
2. `(width, height)` 决定了画布上有多少个像素（即分辨率）
3. `stride` 是画布一行有多少个“像素单元”（由于性能优化的对齐要求，可能略大于 width）
4. `pixel_format` 决定了每个像素的数据如何编码颜色信息（RGB、BGR 等）

当我们向帧缓冲写入数据时，显卡通过扫描帧缓冲的内容，将像素数据转换为 VGA/DVI/HDMI 等视频信号输出到显示器。因此，“绘制图形”的本质就是向帧缓冲中的特定位置写入正确的颜色值。

本实验中，我们通过 `get_handle_for_protocol::<GraphicsOutput>()` 获取 GOP 协议实例，读取帧缓冲的物理地址和显示参数，在退出 UEFI Boot Services 前将这些信息封装为 `GopInfo`，并通过 `BootInfo` 结构体传递给内核。内核收到后，将物理地址转换为虚拟地址（加上 `physical_memory_offset`），从而可以在分页环境下安全地访问帧缓冲内存。基于此，我们实现了逐像素绘制、矩形填充、滚屏等基本操作，并进一步构建了字符渲染引擎、文本控制台和动画时钟。

9. 思考题

9.1. 为什么要在退出 UEFI Boot Services 之前获取 GOP 信息？退出之后还能获取吗？

UEFI 将系统的生命周期分为多个阶段。在 Boot Services 阶段（即内核接手控制权之前），UEFI 提供了一系列协议供 bootloader 使用，包括 GOP（图形输出协议）、文件系统协议、内存分配服务等。一旦调用了 `exit_boot_services()`，系统进入 Runtime Services 阶段，所有 Boot Services 协议都不再可用，相关的内存也会被回收。

因此，必须在 `exit_boot_services()` 之前获取 GOP 信息，并保存帧缓冲的物理地址。退出后，虽然 UEFI 的 GOP 协议不可用，但帧缓冲的物理地址仍然有效——因为显卡依然在扫描那段内存输出画面。只要内核正确地映射了该物理地址并继续向帧缓冲写入数据，画面就可以持续更新。

9.2. 为什么使用 `write_volatile` 而非普通的指针写入？

帧缓冲位于 MMIO（Memory-Mapped I/O）区域，对它的写入是“有副作用”的（会改变屏幕上的像素）。Rust 编译器的优化器可能对普通写入进行合并、重排甚至消除（如果后续有对同一位置的覆盖写入）。使用 `write_volatile` 可以告诉编译器：

1. 每次写入都必须确实发生，不能被优化掉
2. 写入的顺序必须保持代码中的顺序
3. 不能对写入进行合并

9.3. 为什么文本控制台需要保存帧缓冲的副本引用，而不是每次都通过 `VgaDriver` 访问？

主要是为了性能和避免死锁。`VgaDriver` 的访问受互斥锁保护。如果文本控制台每次写字符都要获取 VGA 锁，那么在日志系统（可能在中断上下文中调用）写入 VGA 时可能与 Shell（进程上下文）竞争锁资源，导致死锁或严重的性能下降。

因此 `VgaConsole` 在初始化时保存了帧缓冲的裸指针、宽度、高度和步长，后续所有像素操作都直接通过这些副本进行，完全不经过 `VgaDriver` 的互斥锁。`VgaDriver` 仍保留互斥锁供 `clock_demo` 这类需要独占访问帧缓冲上部分的进程使用。

9.4. 分屏模式下，如何确保时钟进程和文本控制台不会互相干扰?

通过垂直分区约定：时钟进程只读写屏幕上方 `[0, clock_height)` 区域的像素，文本控制台只读写 `[separator_y, total_height)` 区域的像素。两个区域之间有 1 像素高的分隔线，互不重叠。

由于两个区域的像素地址在帧缓冲中是不重叠的，即使两个“线程”同时写入（尽管实际上帧缓冲的字节级写入是原子的，不会有“撕裂”的像素），也不会产生冲突。

9.5. 如果 GOP 协议不可用（如纯串口模式），系统会如何表现?

`get_gop_info()` 在获取协议失败时返回 `None`。内核在 `init()` 中检查 `boot_info.gop_info.is_some()`，如果为 `None` 则跳过 VGA 驱动的初始化。`kernel_main` 中同样检查后才创建时钟进程和初始化 VGA 控制台。因此系统可以优雅降级为纯串口模式，不影响核心功能的运行。